

Experiment No.2

Aim: Creating smart contract using Solidity and Remix IDE

Code:

1. Basic Solidity Program to print your name

Code:

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.0;
```

```
contract PrintName {
```

```
    // This public variable automatically creates a button to view your name
```

```
    string public myName = "Sarathak Agrawal";
```

```
    // This function explicitly returns your name when called
```

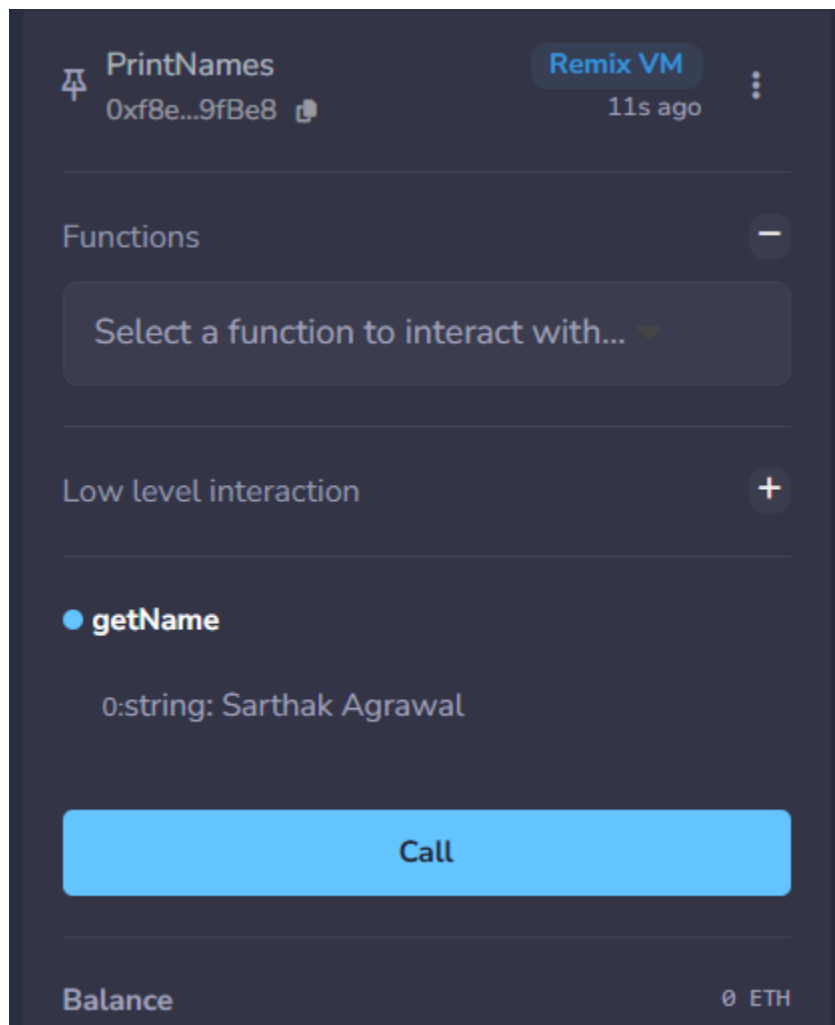
```
    function getName() public view returns (string memory) {
```

```
        return myName;
```

```
    }
```

```
}
```

Output:



2. Use functions to check a number is palindrome or not

Code:

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.0;
```

```
contract PalindromeChecker {
```

```
    // Function to check if a given integer is a palindrome
```

```
    function isPalindrome(uint256 _number) public pure returns (bool) {
```

```
        // Numbers less than 0 are handled by uint256 (only positive integers  
allowed)
```

```
        uint256 originalNumber = _number;
```

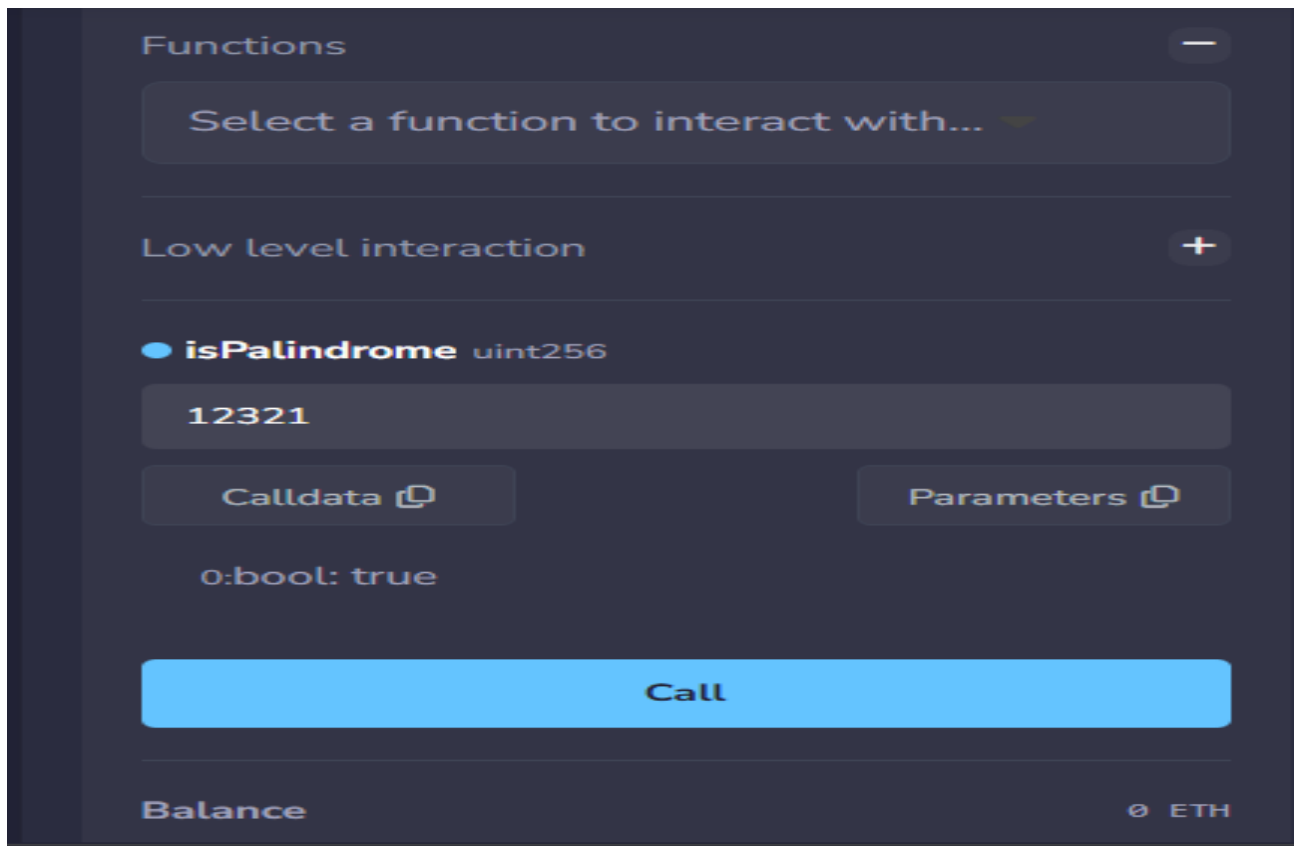
```
        uint256 reversedNumber = 0;
```

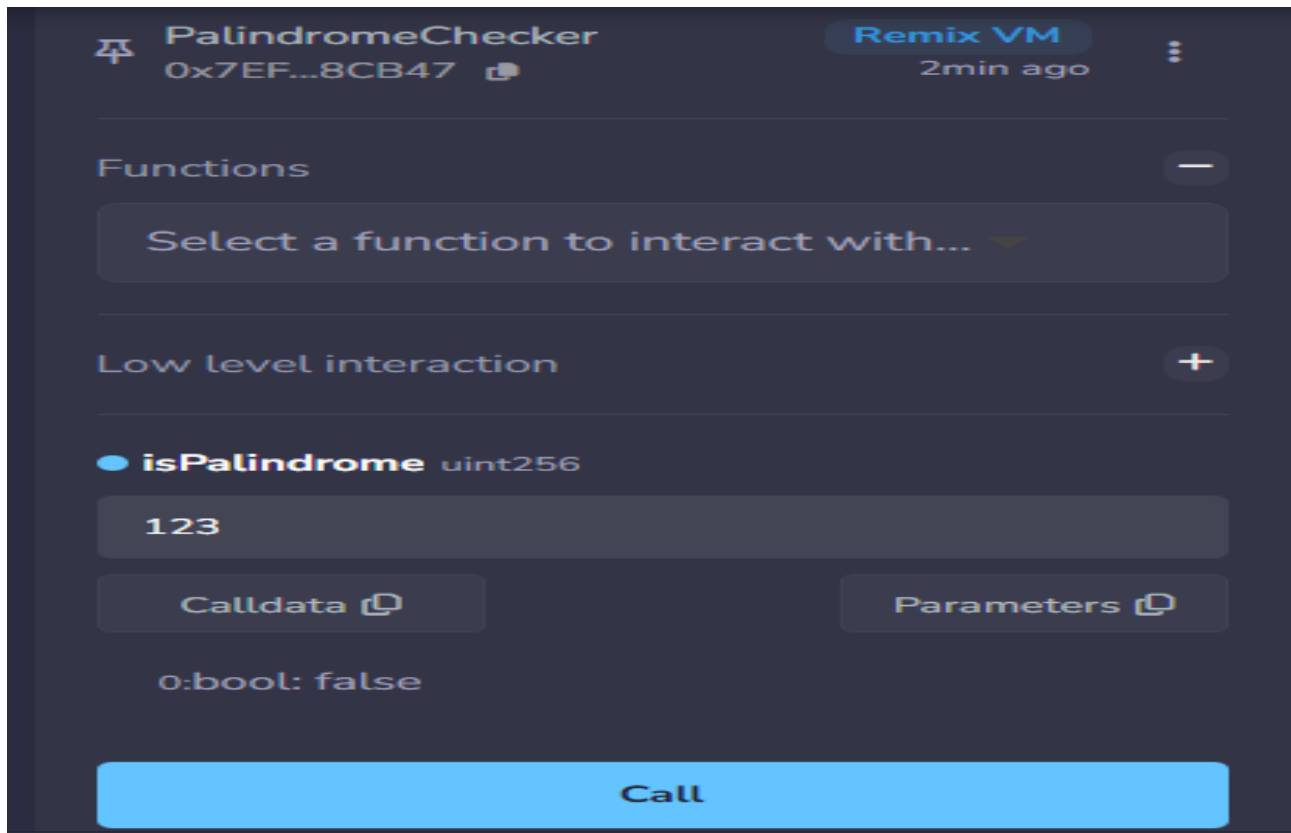
```
        uint256 temp = _number;
```

```
// Loop to reverse the digits of the number
while (temp > 0) {
    uint256 remainder = temp % 10;
    reversedNumber = (reversedNumber * 10) + remainder;
    temp /= 10;
}

// Returns true if the original number matches the reversed number
return (originalNumber == reversedNumber);
}
```

Output:





Code:

3. Create a function `distinct (array, length of array)`. This `distinct ()` will take two arguments: a dynamic `uint` type array and length of array.

The `distinct ()` will return the number of distinct elements in an array.

Code:

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.0;
```

```
contract ArrayUtils {
```

```
    // Function to calculate the number of distinct elements
```

```
    function distinct(uint256[] memory _array, uint256 _length) public pure  
    returns (uint256) {
```

```
        // Edge case: If the array is empty or length provided is 0
```

```
        if (_length == 0 || _array.length == 0) {
```

```
            return 0;
```

```
        }
```

```

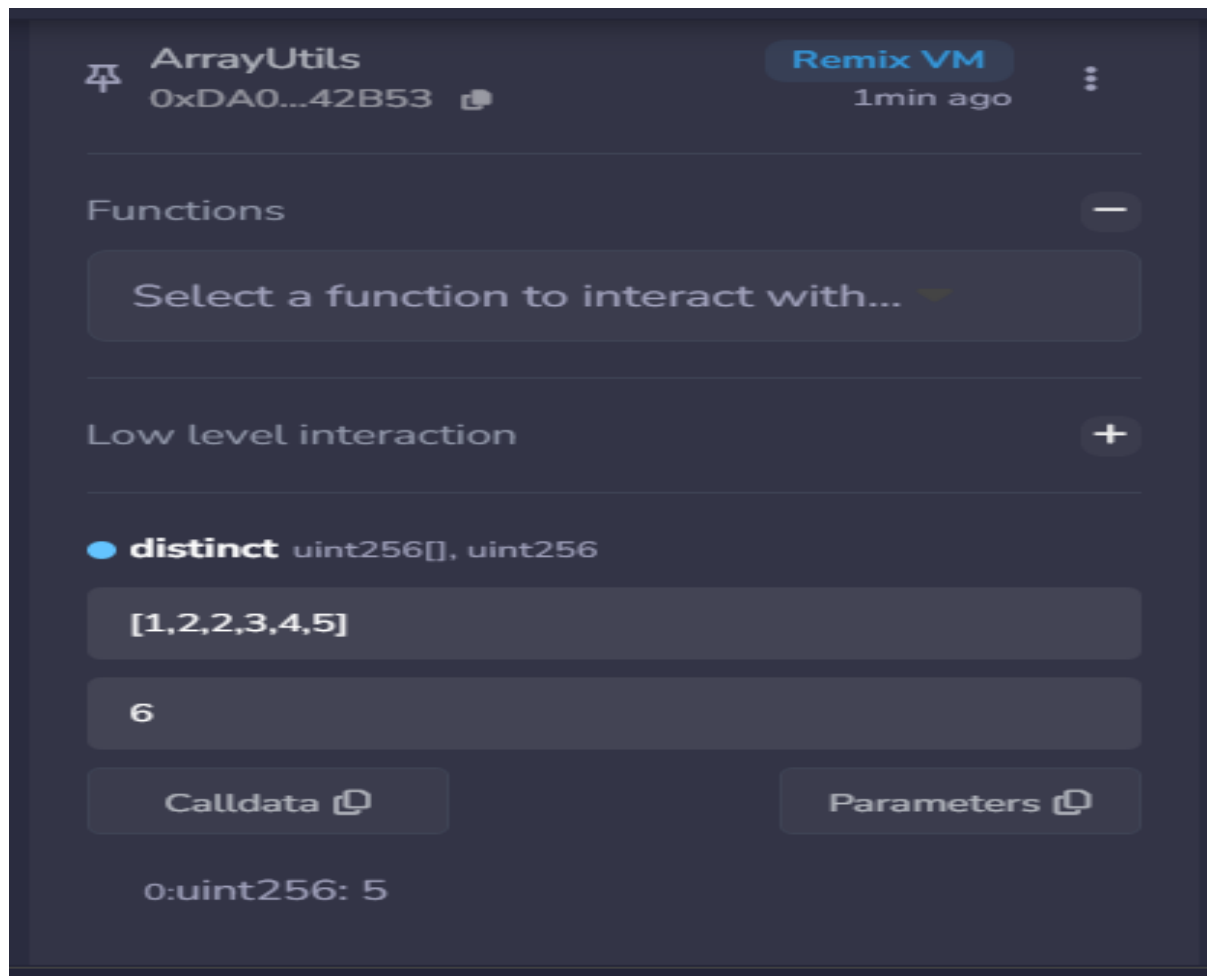
uint256 distinctCount = 0;

// Loop through each element up to the specified length
for (uint256 i = 0; i < _length; i++) {
    bool isDuplicate = false;

    // Check if the current element appeared anywhere before it
    for (uint256 j = 0; j < i; j++) {
        if (_array[i] == _array[j]) {
            isDuplicate = true;
            break; // Found a duplicate, skip further checks for this
element
        }
    }
    // If it hasn't appeared before, it is a distinct element
    if (!isDuplicate) {
        distinctCount++;
    }
}
return distinctCount;
}
}

```

Output:



Code:

4. Concatenation of two strings

CODE:

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.12; // Requires version 0.8.12 or higher for
string.concat()

```
contract StringConcatenation {  
    // Function that takes two strings and returns them joined together  
    function concatenate(string memory _str1, string memory _str2) public  
    pure returns (string memory) {  
        return string.concat(_str1, _str2);  
    }  
}
```

Output:

Select a function to interact with... ▼

Low level interaction +

● concatenate string, string

black

board

Calldata 

Parameters 

0:string: blackboard

Call